

IT3051 · FUNDAMENTALS OF DATA MINING

StayWise

Hotel Booking Cancellation Prediction

Stage 11 — Final Technical Report

Manoharan Subasthican — IT23810778

Kuganesan Kageepan — IT23717572

Gowsika Surendran — IT23794566

Kokulan Seyon — IT23752672

Prepared: 1 October 2026

Historical modelling results, implementation and reproducibility assessment

Contents

Abstract and evidence status	3
1. Introduction and problem definition	4
2. Scenario and stakeholder requirements	4
3. Dataset identification, citation and validation	4
4. Data understanding and EDA	6
5. Cleaning and preprocessing	6
6. Feature engineering and selection	7
7. Algorithms and rationale	8
8. Model comparison and evaluation	8
9. Hyperparameter tuning and optimization	10
10. Final model selection	11
11. System architecture and implementation	12
12. Backend and frontend development	13
13. System testing and results	14
14. Limitations and future improvements	15
15. Individual and group contributions	15
16. Business recommendation	16
References and evidence	17
Appendix A. Recorded application screenshots	18
Appendix B. Reproducibility and acceptance checks	20

Abstract and evidence status

StayWise addresses uncertainty in hotel booking outcomes through supervised binary classification and a local reservation-assessment application. The original dataset contains 119,390 bookings; removing 31,994 exact duplicates leaves 87,396 records with a 27.49% cancellation rate. A stratified 80/20 split and three-fold training cross-validation support the comparison of Logistic Regression, Decision Tree, Random Forest and Extra Trees. Random Forest and Logistic Regression were tuned, with final selection based on training CV ROC-AUC. Tuned Random Forest obtained the highest recorded CV ROC-AUC of 0.9125 and held-out ROC-AUC of 0.9158, accuracy of 85.08%, precision of 75.43% and recall of 67.80%. Feature ablation did not demonstrate a gain from the four engineered fields. The Flask/browser prototype implements input validation, consistent feature derivation and risk display. Earlier documented integration and browser tests passed; the current model artifact fails to decompress, preventing a fresh service-test run and application startup. The report therefore separates recorded modelling results and prior application evidence from current readiness. Temporal validation, calibration, current-hotel evaluation and artifact restoration are the principal next steps.

Report evidence policy

Dataset counts were checked against the local CSV. Model scores and configurations were cross-checked against recorded notebook outputs and the team guide. Application behavior is described from source and earlier testing records. The current artifact decompression failure is reported in Sections 10 and 13; successful current inference is not asserted.

Coverage

Sections 1–15 cover every Stage 11 requirement. Section 16 discusses conclusions and business use; the references and appendices provide traceable evidence and reproducibility instructions.

1. Introduction and problem definition

Hotel cancellations create uncertainty in occupancy, staffing and room allocation. This project estimates whether a reservation will be canceled using historical booking characteristics. The binary target is `is_canceled`: 0 means not canceled and 1 means canceled. StayWise translates this estimate into a clear result and suggested follow-up for hotel staff. No financial saving or operational improvement has been measured in a real hotel deployment.

The project objectives are to identify relevant booking patterns, prepare reproducible inputs, compare at least four suitable classifiers, optimize selected candidates, and integrate the final model into a usable prediction interface. The scope is historical supervised binary classification and a local decision-support prototype. Live reservation-system integration, measured revenue impact and production hosting are outside the implemented scope.

2. Scenario and stakeholder requirements

Reservation staff need to assess a booking quickly, understand missing-input errors and receive a meaningful outcome. Managers and revenue teams need risk information for follow-up and contingency planning. The system must use information available before the final cancellation outcome. The application provides labeled categorical and numeric fields, server-side validation, estimated cancellation probability, likely outcome and a suggested next action. Assigned-room type and booking changes are retained by the trained model, so this is an assessment at a point when those details are available, not necessarily at initial booking creation.

Requirement	Implemented response
Reservation input	Grouped form with 26 raw user fields and labeled choices
Clear assessment	Cancellation/retention scores, binary outcome, risk band and follow-up
Input quality	Server validation, required-field messages and cross-field checks
Training consistency	Ordered 33-column frame and existing fitted pipeline
Usability	Loading state, reset/example actions, responsive layout and JSON download
Operational boundaries	No stored booking database; local demonstration service

A valid prediction point must be defined when operationally integrating this system. Fields such as assigned room and booking changes can be known during a reservation review but may not exist at initial booking. Final status is never an allowed predictor.

3. Dataset identification, citation and validation

The Hotel Booking Demand dataset was obtained through Jesse Mostipak's Kaggle distribution. The research source is Antonio, N., de Almeida, A., and Nunes, L. (2019), Hotel booking demand datasets, Data in Brief, 22, 41–49, DOI 10.1016/j.dib.2018.11.126. Source attribution is taken from the existing project proposal.

The local CSV contains 119,390 rows and 32 columns, covering City Hotel and Resort Hotel bookings with arrivals from July 2015 to August 2017. The original labels are 75,166 not canceled and 44,224 canceled. Validation includes shape/type inspection, missing-value counts, duplicate inspection, target checks, categorical cardinality review and leakage review. Instructor approval is stated in the existing report; an independent signed approval record is not present in this workspace.

Dataset property	Verified value
Original observations / columns	119,390 / 32
Raw target counts	Kept 75,166; canceled 44,224
Exact duplicates	31,994
Clean observations / label counts	87,396; kept 63,371; canceled 24,025
Missing children / country	4 / 488
Missing agent / company	16,340 / 112,593

Dataset suitability follows from a defined binary outcome, booking-level operational predictors, a substantial record count and mixed numeric/categorical inputs. Technical data validation is separate from instructor authorization: approval is reported in existing project documentation, but this report cannot independently certify a signed approval record. Historical country and intermediary codes require careful interpretation; the data are not evidence of current local hotel behavior.

4. Data understanding and EDA

The notebook inspects hotel, month, customer type, market segment, country, room codes, lead time, average daily rate and stay length. EDA identifies class imbalance, missing identifiers, high categorical cardinality, duplicate rows and differences in observed cancellation rates between categories. These are descriptive associations and do not demonstrate causes of cancellation. The original cancellation prevalence is 37.04%, while removing duplicates changes it to approximately 27.49%; the cleaned prevalence is the relevant baseline for model evaluation.

EDA observation	Modelling implication
Raw City cancellation 41.73%; Resort 27.76%	Retain hotel category; avoid a single universal rate
Substantial missing company identifiers	Encode no recorded identifier without discarding most rows
Duplicate removal reduces cancellation prevalence	Use cleaned prevalence when interpreting baseline accuracy
Lead time and requests vary by outcome	Retain as candidate signals; associations are not causal
ADR range -6.38 to 5,400	Investigate anomalies; no documented clipping was applied
Outcome-related reservation fields	Exclude to prevent obvious target leakage

The notebook uses count plots, category-level cancellation rates, histograms, boxplots and a numeric correlation heatmap. Calendar month ordering aids seasonal interpretation. Rates for tiny categories, such as Undefined market segment with two raw rows, should not support broad conclusions. Non Refund deposits have a strong historical association with cancellation, but it does not prove the deposit arrangement causes cancellation. No individual causal explanation or feature-importance ranking is claimed.

5. Cleaning and preprocessing

The notebook removes 31,994 exact duplicates, leaving 87,396 rows. Missing children values are filled using the median (zero in this dataset); country uses Unknown; agent/company use zero and integer normalization. Other categorical defaults include SC for meal and Transient for customer type where applicable. Reservation status and reservation-status date are removed because they expose the final outcome.

The stratified 80/20 split with random seed 42 produces 69,916 training rows and 17,480 test rows. Preprocessing inside each fitted pipeline includes numeric imputation and standardization and categorical imputation and one-hot encoding, with unknown-category handling and minimum category frequency 50. Agent/company are treated as categorical identifiers. Cross-validation fits preprocessing within its training folds. Some preliminary cleaning is performed before splitting, so future work should move all learned cleaning into training-only pipeline steps.

The numeric transformer uses SimpleImputer(strategy='median') followed by StandardScaler. The categorical transformer uses most-frequent imputation and OneHotEncoder(handle_unknown='ignore', min_frequency=50). Scaling is particularly relevant to the regularized linear baseline; sharing preparation across algorithms keeps the comparison consistent. Unknown categories do not cause an encoder error, although the current application restricts its form to known options.

Training labels are 50,696 kept and 19,220 canceled; test labels are 12,675 kept and 4,805 canceled. Stratification preserves class proportions. Three-fold StratifiedKFold uses shuffle=True and random_state=42. The preliminary processed training array is not the Stage 6 modelling input: each candidate receives a cloned preprocessing template within its own pipeline. No SMOTE or class resampling was applied. Balanced weights were explored only in Logistic Regression tuning.

6. Feature engineering and selection

The model uses 33 features: 21 numeric and 12 categorical. Engineered features are total nights = week nights + weekend nights; long stay = total nights >7; weekend stay = weekend nights >0; high lead time = lead time >60. Feature exclusion removes the outcome fields and the target from predictors. Ablation gives training CV ROC-AUC 0.9125 with engineering and 0.9126 without it. This difference of approximately -0.0001 provides no evidence of improvement from engineered features. They remain in the supplied artifact and are reproduced exactly in the application.

Derived field	Definition	Boundary behavior
total_nights	Week nights + weekend nights	Sum of planned stay
is_long_stay	total_nights > 7	Seven nights gives 0; eight gives 1
is_weekend_stay	Weekend nights > 0	Zero gives 0; positive gives 1
is_high_lead_time	lead_time > 60	Sixty days gives 0; sixty-one gives 1

Boolean indicators become integers. These thresholds are chosen rules, not learned optimal cutoffs. total_guests was proposed in early documents but is not a modelling feature in the current notebook. No automated feature-selection algorithm is documented. The ablation holds selected classifier settings and validation folds fixed rather than retuning each representation.

7. Algorithms and rationale

Logistic Regression provides a linear baseline; Decision Tree captures simple nonlinear decision rules; Random Forest combines trees to capture interactions with improved stability; Extra Trees introduces further randomization as an ensemble comparison. All candidates are compared using the same training-only three-fold stratified cross-validation. ROC-AUC is the selection criterion because it measures discrimination across thresholds. Accuracy alone is insufficient: always predicting no cancellation gives approximately 72.51% on the cleaned/test population.

Algorithm	Rationale	Baseline configuration
Logistic Regression	Regularized linear classification reference	max_iter=1000; seed 42
Decision Tree	Nonlinear rule/interaction reference	max_depth=20; min_samples_leaf=5; seed 42
Random Forest	Ensemble of fitted trees for flexible discrimination	120 trees; min_samples_leaf=2; seed 42
Extra Trees	Ensemble with additional split randomization	120 trees; min_samples_leaf=2; seed 42

Random Forest and Extra Trees use `n_jobs=-1`. The measured ensemble advantage is consistent with useful nonlinear structure, but score differences alone do not establish a precise causal explanation. A single tree can have greater variance; constrained depth and leaf size control complexity without guaranteeing generalization.

8. Model comparison and evaluation

Training CV ROC-AUC results saved in the notebook:

Candidate	CV ROC-AUC
Tuned Random Forest	0.9125
Baseline Random Forest	0.9097
Baseline Extra Trees	0.9058
Tuned Logistic Regression	0.8644
Baseline Logistic Regression	0.8640
Baseline Decision Tree	0.8476

The selected model's saved held-out results are accuracy 85.08%, balanced accuracy 79.72%, precision 75.43%, recall 67.80%, F1 71.42%, ROC-AUC 0.9158, average precision 0.8123, specificity 91.63% and MCC 0.6152.

Actual outcome	Predicted kept	Predicted canceled
Not canceled	11,614	1,061
Canceled	1,547	3,258

The model identifies 3,258 of 4,805 actual cancellations and misses 1,547. Of the flagged bookings, 1,061 actually remain. Staff should therefore use predictions to prioritize contact and planning, rather than treat them as certainty. The notebook includes confusion-matrix, ROC and precision-recall plots.

Baseline	AUC mean / SD	Accuracy	Precision	Recall	F1
Random Forest	.9097 / .0015	.8474	.7887	.6075	.6863
Extra Trees	.9058 / .0017	.8435	.7802	.5994	.6779
Logistic Regression	.8640 / .0026	.8062	.6893	.5372	.6038
Decision Tree	.8476 / .0025	.8166	.6744	.6439	.6587

The train-minus-validation AUC gap is 0.0762 for Random Forest, 0.0811 for Extra Trees, 0.0027 for Logistic Regression and 0.1144 for Decision Tree. Larger gaps indicate overfitting concerns, but the primary ranking uses validation performance. Fold standard deviations describe fold variability; they are not confidence intervals.

Cancellation is the positive class. Precision = $TP/(TP+FP)$, recall = $TP/(TP+FN)$, and F1 is their harmonic mean. Accuracy measures all correct labels; balanced accuracy averages the two class recalls. ROC-AUC measures discrimination across thresholds rather than percent correct. Average precision summarizes the precision-recall curve and complements AUC under imbalance. The default classification threshold is 0.5. Test accuracy improves on the all-kept baseline by about 12.57 percentage points; no profit improvement has been measured.

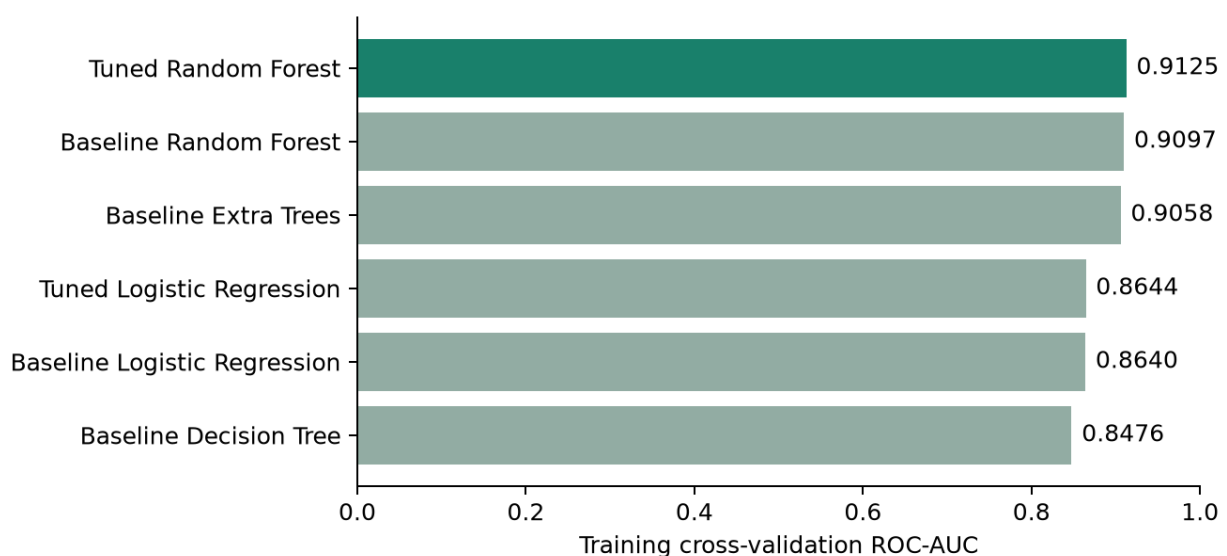


Figure 1. Baseline and tuned training CV comparison. Values are recorded means, rounded to four decimals; axis starts at zero.

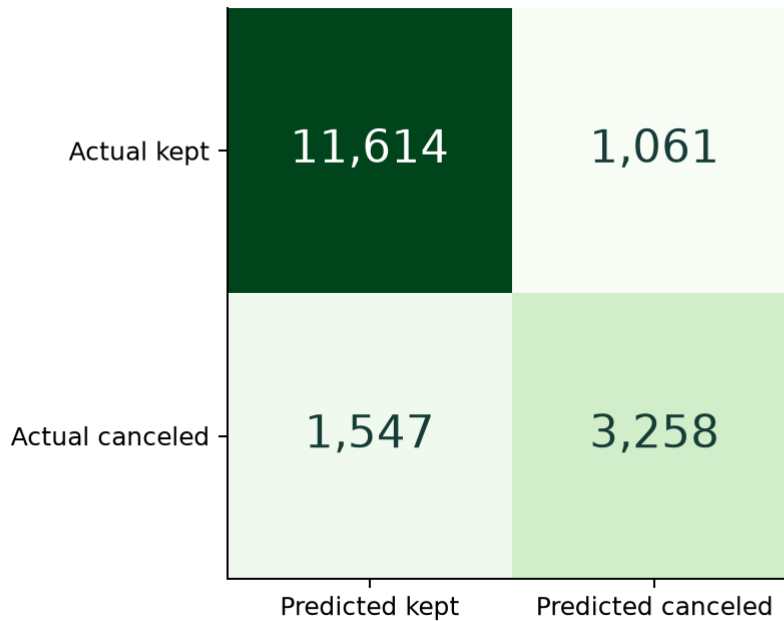


Figure 2. Recorded held-out confusion matrix at threshold 0.5 (17,480 bookings).

9. Hyperparameter tuning and optimization

Random Forest uses a four-candidate randomized search across tree count, depth, leaf size and feature fraction. The selected parameters are 100 trees, depth 20, minimum leaf size 2 and maximum feature fraction 0.7. Logistic Regression uses grid search across C values 0.3, 1 and 3 and class weights None or balanced; its best result uses C=3 and balanced weights. Both searches use the same cross-validation ROC-AUC criterion. The limited search budget is an optimization limitation.

Search	Parameter space	Budget and best result
RandomizedSearchCV / Forest	Trees {100,160}; depth {None,20}; leaf {2,5,10}; features {sqrt,0.7}	4 of 24 combinations; CV AUC .9125
GridSearchCV / Logistic	C {0.3,1,3}; weight {None,balanced}	All 6 combinations; CV AUC .8644

Each candidate is assessed on three folds using ROC-AUC. Searches set refit=True and n_jobs=1; Forest parallelism is internal. Forest randomized sampling uses seed 42. The budgets imply 12 and 18 CV fits respectively, plus best-candidate refitting. Larger C weakens Logistic Regression regularization; balanced weighting modifies loss weights, not the number of examples.

Forest improves from .9097 to .9125 (+.0028) and Logistic Regression from .8640 to .8644 (+.0004). The same CV supports parameter selection and candidate comparison, so selected CV estimates can be optimistic. The independent held-out partition provides the final evaluation. Larger searches, nested validation and uncertainty analysis were not completed.

10. Final model selection

Tuned Random Forest was selected using the highest mean training cross-validation ROC-AUC (0.9125), then refitted on all 69,916 training rows. The test partition was used to evaluate this selected candidate, not to select the winner. Its test ROC-AUC is 0.9158 and accuracy is 85.08%. The modest improvement over baseline Random Forest is 0.0028 CV ROC-AUC; statistical significance was not established.

The notebook export stores a fitted deployment pipeline, the selected model name, target name, ordered feature columns, classification threshold 0.5, training CV ROC-AUC and held-out test metrics. Identifier normalization uses built-in pandas operations in FunctionTransformer steps, followed by the fitted preprocessing/classifier pipeline. The notebook contains reload-verification statements and recorded export evidence; no custom helper module is required by this implementation.

During the report preparation check on 1 October 2026, the current Model/best_hotel_cancellation_model.joblib could not be loaded: zlib.error: Error -3 while decompressing data: invalid block type. This establishes that the current file is unreadable in the project environment; its cause was not determined. Recorded notebook results remain the basis for model evaluation in this report. Successful loading of the current file, current probability parity and current end-to-end inference are not claimed.

11. System architecture and implementation

The browser requests the Flask-rendered form and static CSS/JavaScript. JavaScript submits booking JSON to POST /api/predict on the same origin. The service checks field values, constructs the ordered feature frame, calls the saved pipeline and returns structured JSON. The browser displays the result without a page reload. GET /api/health reports readiness; GET /api/model exposes model information and an example payload. Inputs are processed in memory and are not stored in a database. Optional JSON downloads are created in the user's browser.

Architecture: Browser form → Flask input validation → calendar/feature derivation → saved normalization and preprocessing → Random Forest → JSON outcome and risk → browser assessment.

Component	Responsibility
Model/new.ipynb	EDA, preparation, experiments, tuning, evaluation and export
Model/*.joblib	Serialized fitted pipeline and prediction metadata
app.py	Model loading, validation, input derivation and JSON endpoints
templates/index.html	Server-rendered booking interface
static/app.js / style.css	Interactive assessment behavior and responsive styling
tests/test_app.py	Service contract, edge cases, failures and parity checks

The training lifecycle is separate from inference. Inference applies already fitted transformations and calls predict_proba; it does not fit on new booking input. Categories and displayed metrics are read from the artifact, reducing duplication. At present artifact loading happens at module import, which explains why a damaged artifact blocks startup rather than producing a health endpoint response.

Browser form / 26 raw fields
Flask validation and calendar / feature derivation
Ordered frame / 33 predictors
Fitted identifier normalization and preprocessing
Tuned Random Forest / predict_proba
JSON outcome, probabilities, risk band and follow-up

Figure 3. Inference flow, top to bottom; responses return to the browser. The current artifact-load defect blocks startup before this flow can execute.

12. Backend and frontend development

The backend uses Flask and loads `Model/best_hotel_cancellation_model.joblib` relative to `app.py`, allowing launch from any working directory. It validates required fields, numeric finiteness, integer counts, nonnegative operational values, dates and known categorical options. Optional children, country, agent and company use training-compatible defaults. It rejects zero guests and zero nights for the operational form; this narrows the accepted domain compared with the historical dataset. Validation upper bounds are application safeguards, not statistical training ranges.

Arrival date produces year, month name, ISO week and day of month. The four derived features use the notebook's exact rules. Agent/company stay integral before the artifact converts them to strings. The fitted pipeline applies imputation, scaling and encoding. Results include both class probabilities, outcome, risk band, threshold, recommended action and limitations. HTTP errors are 400 for invalid input, 415 for wrong content type, 413 for oversized inputs and 503 for inference failure; internal exception details are logged rather than sent to users.

The frontend groups 26 user fields into booking/arrival, guests/preferences, arrangements, and history. It includes a demonstration example, reset, loading indicator, accessible labels, field-specific errors, mobile layout and downloadable JSON assessment. It disables editing while inference runs and clears previous predictions on edits. Performance cards show saved test metrics. Low (<25%), Moderate (25–<50%) and High (≥50%) bands are operational display conventions; they have not been optimized for business cost or calibrated. The UI links scores to proportionate follow-up suggestions without claiming individual causal explanations.

Endpoint	Purpose	Expected response
GET /	Booking interface	HTML
GET /api/health	Readiness and selected model	JSON
GET /api/model	Model metrics, categories and example	JSON
POST /api/predict	Validate and assess a booking	JSON result or field errors

The request body is limited to 32 KiB. Integer operational counts reject fractional values and booleans; numeric values must be finite and within configured bounds. Arrival dates must parse as ISO dates with years between 1900 and 2100. ADR is a nonnegative finite number in the operational form even though historical records include a negative value. Optional defaults cover children, country, agent and company; remaining fields are required.

The service runs on 127.0.0.1:5000 with Flask debug disabled for the local demonstration. `requirements.txt` pins the application dependencies. `README.md` documents Python 3.13 setup, local launch and test commands for Windows and macOS; actual physical macOS execution is not verified. There is no production authentication, persistent audit database or external reservation integration in the implemented prototype.

13. System testing and results

Two evidence periods must be distinguished. SYSTEM_TEST_RESULTS.md records an earlier successful local run on 1 October 2026: seven automated integration tests, prediction parity for 20 historical rows within 1e-12, live HTTP checks, and desktop/mobile Chromium checks. Screenshots saved from that earlier run are included in Appendix A. These records describe software correctness and integration; they do not measure new predictive accuracy.

The fresh report-preparation run of python -m unittest discover -s tests -v failed during test-module import. tests/test_app.py imports app.py, which loads the model at module initialization. The joblib decompression error prevents application import, so none of the seven service tests could execute. The loader reports one import error; it is not one failed prediction assertion. The current application cannot start through its normal entry point while this artifact remains unreadable. The request-time 503 handler cannot catch this startup error.

Check	Earlier recorded evidence	Current report-preparation check
Seven integration tests	Passed	Blocked by model-load import error
Historical-row parity	20 rows; tolerance 1e-12	Not rerun successfully
Desktop/mobile browser	Passed at 1440 x 1000 and 390 x 844	Earlier screenshots only
JavaScript syntax	Passed	node --check static/app.js passed
Dataset audit	Counts recorded	Shape, duplicates, missing values and labels reconfirmed
Model artifact load	Previously recorded as verified	Decompression failure reproduced

Existing integration coverage includes page/readiness, model information, probability range and complementary scores, class/threshold agreement, missing fields, invalid dates/categories, negative/fractional/nonfinite values, optional defaults, zero guests/nights, feature thresholds at 60/61 days and 7/8 nights, body-size limits and controlled inference failure. Protocol expectations are 400 for malformed/invalid input, 415 for non-JSON, 413 for oversized requests and 503 for request-time inference errors.

The earlier browser record includes a 24.97% cancellation estimate for the example booking, assessment download, clearing stale results after edits, a zero-guest error, reset behavior and no horizontal overflow or runtime errors. This example is illustrative and must not be presented as a current prediction until the artifact is restored.

Release readiness requires restoring a known-good artifact or re-exporting from the notebook, confirming its metadata and prediction behavior, rerunning all seven integration tests, and repeating desktop/mobile browser checks. The original failing artifact was not altered as part of report generation.

14. Limitations and future improvements

The data are historical and represent two hotels; present-day local hotels can differ. Random splitting does not establish performance on a later time period. Room assignments and booking updates need a clearly defined prediction point. Country, agent and company categories may have location-specific meaning. Arrival-week conventions should be verified with any external reservation system. The application restricts unknown categorical inputs while the model encoder can technically handle unseen categories. Model probabilities are not calibrated and the threshold is not cost-optimized. Group-specific fairness and live business impact have not been assessed.

Future improvements include temporal/external validation, current hotel data, calibration, cost-based thresholds, subgroup analysis, booking-system integration, drift monitoring and a production WSGI deployment with appropriate access controls. These are proposed extensions, not completed features.

Prioritize improvements in this order: restore artifact integrity and regression checks; validate a precise prediction-time feature set; perform time-based and external-hotel evaluation; assess calibration and intervention costs; then pilot current data with subgroup and drift monitoring. Country/intermediary identifiers can proxy location or customer groups, so subgroup assessment should precede consequential automated policies. Access controls, secure hosting and an appropriate data-retention policy are needed before handling live reservation information. Current in-memory processing avoids a stored booking database, but it does not alone establish production privacy compliance.

15. Individual and group contributions

The team comprises the four members identified in the proposal and TEAM_VIVA_GUIDE.md. The allocation below records documented study and explanation responsibilities. The workspace does not establish authorship of each code statement, so it is not presented as an independently verified contribution log.

Member / ID	Documented responsibility
Manoharan Subasthican / IT23810778	Categorical EDA, leakage, Logistic Regression/tuning, comparison and model selection
Kuganesan Kageepan / IT23717572	Numeric/room EDA, engineered features, Decision Tree, held-out metrics and plots
Gowsika Surendran / IT23794566	Data structure, missingness, duplicates, cleaning, Random Forest/tuning and ablation
Kokulan Seyon / IT23752672	Country/cardinality, preprocessing/split, pipeline setup, Extra Trees and artifact export

Each member should be able to explain the complete workflow and substantiate actual personal contributions using code, experiments or other records. The spelling Kageepan follows the formal project documents. The guide uses Kageepan as a short name.

Application/report/presentation work in the workspace was developed with coding-assistant support. This assistance is acknowledged here; members remain responsible for checking the work and following their institution's disclosure rules. Shared responsibilities include reviewing the consistency of

preparation and inference, interpreting model errors and presenting limitations honestly.

16. Business recommendation

The project implemented a complete data-mining workflow and application source for hotel booking cancellation assessment. Four baseline algorithms were compared, two algorithms were tuned, and the final classifier was selected using training-only cross-validation. Tuned Random Forest produced the strongest recorded validation ranking, and its held-out evaluation shows useful historical discrimination with material false positives and missed cancellations.

Reservation teams could use this estimate to prioritize courteous confirmation and account for uncertainty in occupancy planning. Predictions should support staff judgement, not automatically reject guests. A monitored local pilot should establish current-hotel performance, appropriate follow-up costs and measured operational value before policies depend on the scores.

The modelling evidence supports the reported historical findings. The application source, tests and earlier browser evidence support the implemented design. The current unreadable artifact remains a reproducibility and startup defect; consequently, present-day executable readiness is not established. Resolving that defect and completing regression checks are the immediate next steps before demonstration or deployment.

References and evidence

[1] Antonio, N., de Almeida, A., and Nunes, L. (2019). Hotel booking demand datasets. Data in Brief, 22, 41–49. <https://doi.org/10.1016/j.dib.2018.11.126>. Bibliographic details follow the local project proposal.

[2] Mostipak, J. Hotel Booking Demand. Kaggle distribution: <https://www.kaggle.com/datasets/jessemostipak/hotel-booking-demand>. Download source recorded in the project proposal; the CSV used is hotel_bookings.csv.

[3] IT3051 Fundamentals of Data Mining, Mini Project Assignment 2026. Local document: doc/IT3051_FDM_Mini_Project_Assignment_2026.pdf. Stage 11 specifies the technical-report coverage.

[4] Project proposal and existing report: doc/FDM.pdf; doc/IT3051_Hotel_Booking_Dataset_Proposal_Expanded.pdf; doc/IT3051_Final_Technical_Report_Hotel_Booking.pdf. Sources for scenario, team identities and reported instructor validation.

[5] Project experiment evidence: Model/new.ipynb and TEAM_VIVA_GUIDE.md. Sources for model configurations, recorded full-data CV/held-out outputs and study allocation.

[6] Implementation and testing evidence: app.py; templates/index.html; static/app.js; static/style.css; tests/test_app.py; tests/browser_smoke.cjs; doc/SYSTEM_TEST_RESULTS.md; README.md. Current import failure and dataset checks were reproduced during this report's preparation on 1 October 2026.

Source practice: this report draws on local project evidence. Published citations and source URLs are reproduced from the proposal; new online source verification was not performed. Earlier test evidence and screenshots are explicitly distinguished from current executable readiness.

Appendix A. Recorded application screenshots

The following screenshots were saved from the earlier documented browser checks. They are evidence of the interface at that time, not a fresh successful run with the current artifact.

StayWise
RESERVATION INTELLIGENCE

HOTEL OPERATIONS

- ◆ Booking assessment
- 🔍 Model performance

Workspace / Booking assessment

Trained model connected

PLAN WITH MORE CONFIDENCE

Every booking tells a story.

Assess cancellation risk and give your team a clearer next step.

Load example booking

Test accuracy

85.1%

On 17,480 held-out bookings

Risk ranking · ROC-AUC

0.916

Higher scores indicate better ranking

Cancellation detection · recall

67.8%

Actual cancellations identified

Booking details

Use the details known at the time of assessment.

01 / INPUT

All fields are required except children, country, agent and company. Room codes and agent IDs should match your reservation records. BB = breakfast, HB = half board, FB = full board, SC = self catering; TA/TO = travel agent / tour operator.

1 Booking & arrival

Hotel type

City Hotel

Arrival date

dd/mm/yyyy

Days between booking and arrival

Week nights (Mon–Fri)

Weekend nights (Sat–Sun)

2 Guests & preferences

Adults

Children · optional

Risk assessment

02 / RESULT

A clearer view of your booking

Complete the booking details to see the estimated cancellation risk and recommended follow-up.

- 1 Enter reservation details
- 2 Assess cancellation risk
- 3 Plan your next action

Support better decisions

Prediction service ready
Hotel cancellation decision support

Figure A1. Desktop booking form.

PLAN WITH MORE CONFIDENCE

Every booking tells a story.

Assess cancellation risk and give your team a clearer next step.

↗ Load example booking

Test accuracy

85.1%

On 17,480 held-out bookings

Risk ranking · ROC-AUC

0.916

Higher scores indicate better ranking

Cancellation detection · recall

67.8%

Actual cancellations identified

Booking details

Use the details known at the time of assessment.

01 / INPUT

All fields are required except children, country, agent and company. Room codes and agent IDs should match your reservation records. BB = breakfast, HB = half board, FB = full board, SC = self catering; TA/TO = travel agent / tour operator.

1 Booking & arrival

Hotel type

City Hotel

Arrival date

06/10/2026

Days between booking and arrival

30

Week nights (Mon–Fri)

3

Weekend nights (Sat–Sun)

1

2 Guests & preferences

Adults

Children · optional

Risk assessment

02 / RESULT

Low cancellation risk

24.97%

estimated cancellation probability

Likely to keep booking

City Hotel · 4 nights · Arrival 2026-10-06

SUGGESTED NEXT STEP

Continue routine reservation follow-up and occupancy planning.

An estimated risk, not a guarantee. Risk bands are operational guidance; probabilities have not been calibrated. Training bookings date from 2015–2017.

↓ Download assessment

Figure A2. Desktop assessment output.

Appendix B. Reproducibility and acceptance checks

Use Python 3.13 and the application dependencies pinned in requirements.txt. Do not copy an existing Windows virtual environment to another computer. Retain app.py, the Model directory, templates, static assets and tests together; retain hotel_bookings.csv for historical-row parity checks.

Step	Command / action	Acceptance evidence
Environment	<code>python -m pip install -r requirements.txt</code>	Dependencies installed
Artifact recovery	Restore a known-good export or run notebook export after full preparation/training	joblib.load succeeds; required metadata exists
Service tests	<code>python -m unittest discover -s tests -v</code>	All seven tests pass
JS syntax	<code>node --check static/app.js</code>	Successful exit
Local launch	<code>python app.py</code>	Service listens on 127.0.0.1:5000
Manual/browser review	Load example, predict, edit, reset, invalid input, download; desktop and mobile	Correct behavior and no runtime errors

Run Model/new.ipynb top to bottom when recreating the model: the later cells depend on the earlier cleaned data and split. Do not substitute reduced smoke-test scores for full-data results. Re-exporting must preserve identifier normalization and the ordered feature contract used by app.py. If new training results differ, update report tables and UI metadata rather than assuming equivalence.

For browser automation, tests/browser_smoke.cjs documents the optional Playwright dependency setup. Production hosting, external hotel-system integration and live operational benefit are not part of the current acceptance evidence.

Current verification summary: CSV shape 119,390 x 32; duplicates 31,994; cleaned target counts 63,371 / 24,025; missing counts children 4, country 488, agent 16,340 and company 112,593. JavaScript syntax passed. The service suite is blocked by the reproduced artifact decompression failure.